Faculty of Sciences of Sfax

WEDTECT

Summer Internship Project:

AquaLip, a Mini Aquaponic System with LoRaWAN Implementation

Internship Report

Mohamed Yassine Abid
Ahmed BenSalah
Yassin Ayedi
Nessryne Chouchene

Supervised by
Manel Elleuchi

Contents

Abstract

This report details the development of a mini aquaponic system, an agricultural technology that provides significant results in limited spaces by recirculating water and nutrients through the symbiosis of fish farming and hydroponic plant cultivation. To optimize water quality for plants and maintain healthy conditions for fish, continuous monitoring of various water parameters is essential, enabling farmers to take appropriate management actions. Based on this need, an Internet of Things (IoT) system was designed to monitor water conditions in aquaponic fishponds, store parameter data in a database, and visualize it via a dedicated website.

The system leverages LoRaWAN[3]ppp technology for long-range, low-power communication. Sensor nodes, built around the Arduino Mega 2560 microcontroller equipped with LoRa shields, measure critical water parameters including temperature (DS18B20), pH (E201-BNC + PH4502C Kit), water level (HC-SR04 ultrasonic sensor), total dissolved solids (TDS SEN0244 DFROBOT), and light intensity (LDR with LM393). A Raspberry Pi with a LoRa HAT serves as the gateway, forwarding data to the ChirpStack network server. The processed data is then presented on a custom-developed website and dashboard, facilitating real-time monitoring and historical data analysis. The results demonstrate that this integrated system functions as expected, significantly simplifying water management in aquaponic farming environments.

1 Introdution

Aquaponics[1], a sustainable food production system, integrates aquaculture (raising fish) and hydroponics (growing plants without soil) in a symbiotic, closed-loop environment. This method offers significant advantages over traditional farming, including reduced water consumption and the elimination of chemical fertilizers. The project detailed in this report aims to develop a mini aquaponic system and enhance its efficiency and monitoring capabilities through the integration of LoRaWAN (Long Range Wide Area Network) technology.

This initiative originated from the I2I hackathon, where the concept of a mini aquaponic system was brainstormed and subsequently awarded, leading to an internship opportunity at WEDTECT Startup, located in Hay Ons Pepiniere. During this internship, the project was systematically divided into several key tasks: researching and selecting appropriate sensors, modeling the system's circuit, developing a website for project overview and demonstration, and implementing a dashboard for real-time sensor data visualization. The chosen server for data management is ChirpStack, and the website serves as the primary application interface. The communication infrastructure relies on a Raspberry Pi equipped with a LoRa HAT acting as the gateway, while the sensor nodes are built using Arduino Mega 2560 microcontrollers paired with LoRa shields, all operating under the LoRaWAN protocol.

2 Background

2.1 Overview

Aquaponics is an innovative and sustainable agricultural practice that combines the cultivation of aquatic animals (aquaculture) with the cultivation of plants in water (hydroponics) in a single, integrated system. This closed-loop system operates on a symbiotic relationship where the waste produced by fish serves as a nutrient source for the plants, and in turn, the plants filter and purify the water, which is then recirculated back to the fish tank. This natural biological filtration process mimics ecosystems found in nature, making it an environmentally friendly and resource-efficient method of food production.

One of the primary benefits of aquaponics is its remarkable water efficiency. Compared to traditional soil-based agriculture, aquaponics can use up to 90% less water because water is continuously recycled within the system, minimizing evaporation and runoff. Furthermore, the system eliminates the need for synthetic chemical fertilizers, as plant nutrients are derived directly from fish waste. This not only reduces environmental pollution but also allows for the production of organic, chemical-free produce and fish.

At the heart of the aquaponic system is the nitrogen cycle. Fish excrete ammonia, which is toxic to them in high concentrations. Beneficial nitrifying bacteria, primarily residing in a biofilter and on the surfaces of the grow media, convert this ammonia first into nitrites and then into nitrates. Nitrates are a less toxic form of nitrogen and are readily absorbed by plants as essential nutrients for their growth. As plants absorb these nitrates, they effectively clean the water, making it safe for the fish to thrive. This continuous cycle ensures a balanced and self-sustaining ecosystem.

Key components typically found in an aquaponic setup include:

- **Fish Tank:** The primary enclosure for the aquatic animals. The type and number of fish depend on the system's size and goals. areas where plants are cultivated. Different methods exist, such as Media Beds (filled with inert media like clay pebbles or gravel, often doubling as a biofilter), or Nutrient Film Technique (NFT) (a thin film of nutrient-rich water flows over the plant roots in channels).
- **Water Pumps:** Essential for circulating water from the fish tank to the grow beds and back, ensuring continuous nutrient delivery to plants and water purification for fish.
- **Air Pumps :** Used to oxygenate the water in both the fish tank and the biofilter, which is crucial for the health of fish and the activity of aerobic nitrifying bacteria.
- **Biofilter:** A dedicated component, especially in larger or more complex systems, designed to provide a large surface area for the colonization of nitrifying bacteria. It plays a critical role in converting fish waste into plant-usable nutrients.
- **Plumbing:** A network of pipes, fittings, and valves that connect all the components, facilitating water flow and system management.

Maintaining optimal water quality parameters is crucial for the success of an aquaponic system. These parameters include pH levels (ideally between 6.8 and 7.2), dissolved oxygen levels, and concentrations of ammonia, nitrites, and nitrates. Regular monitoring and adjustment of these parameters are necessary to ensure the health and productivity of both the fish and the plants.

2.2 LoRaWAN Technology

LoRaWAN (Long Range Wide Area Network)[2] is a low-power, wide-area networking protocol designed for wireless, battery-operated devices in regional, national, or global networks. It is specifically optimized for Internet of Things (IoT) applications that require long-range communication, low power consumption, and secure data transmission. LoRaWAN operates in unlicensed radio frequency bands and enables communication over distances ranging from several kilometers in urban areas to tens of kilometers in rural environments.

The architecture of a LoRaWAN network typically consists of several key elements:

- **End-Devices (Nodes):** These are the sensors or actuators that collect data or perform actions. In an aquaponics context, these would be the pH, temperature, dissolved oxygen, or water level sensors, often connected to microcontrollers like the Arduino Mega 2560 with a LoRa shield.
- **Gateways (Concentrators):** Gateways receive data from end-devices and forward it to a network server. They act as a transparent bridge, converting RF packets from end-devices into IP packets and sending them to the back-end network. The Raspberry Pi with a LoRa HAT serves this role in our system.
- **Network Server:** The network server manages the entire LoRaWAN network. It handles duplicate packet detection, schedules acknowledgment messages, performs adaptive data rate optimization, and routes data to the appropriate application servers. ChirpStack is used as the network server in this project.
- **Application Server:** This server processes the data received from end-devices and provides it to the end-user application. It can also send downlink messages to end-devices. In our case, this is integrated with our custom-developed website and dashboard.

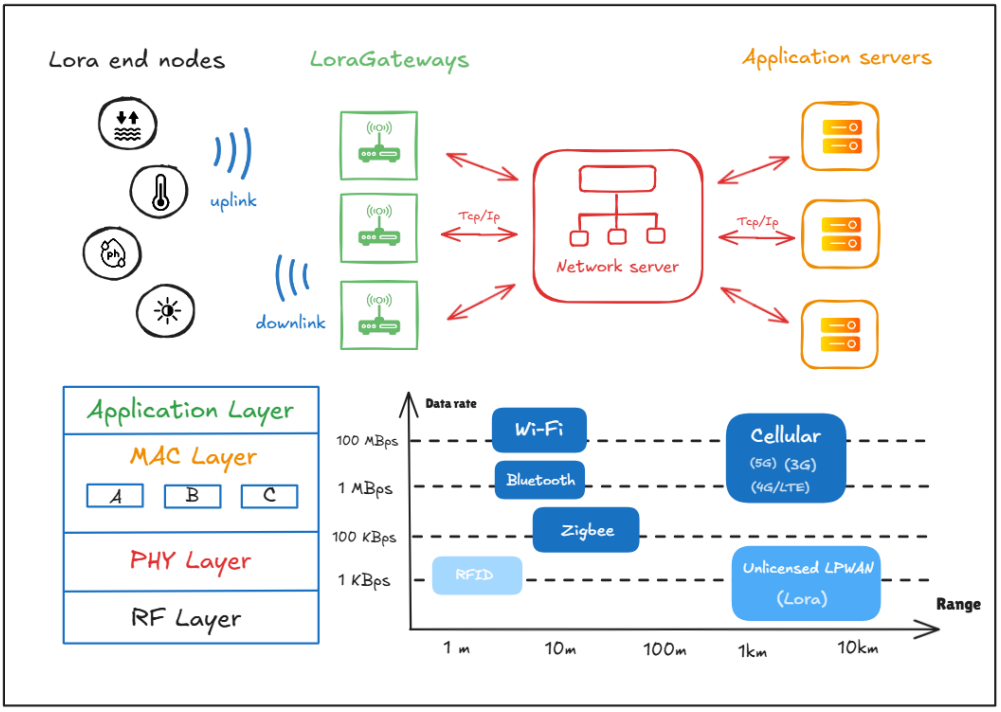

Figure 1: Overall LoRaWan Architecture

3 System Design and Implementation

3.1 Overall System Architecture

The mini aquaponic system with LoRaWAN integration is designed as a comprehensive IoT solution for automated monitoring and control. The architecture follows a typical star-of-stars topology inherent to LoRaWAN, with end-devices communicating with gateways, which then forward data to a central network server and subsequently to an application server.

3.2 Sensor Selection and Circuit Modelisation

Accurate and reliable data collection is paramount for effective aquaponics management. Based on the critical parameters identified for aquaponics, the following sensors were selected and tested:

- **Temperature Sensor (DS18B20):** To track water temperature, which influences metabolic rates of fish, plant growth, and bacterial activity. Available from Didactico, 2BTrading, seli.tn, and Cothings for approximately 6.70 TND.
- **pH Sensor (E201-BNC + PH4502C Kit):** To monitor the acidity/alkalinity of the water, ensuring it remains within the optimal range for fish and plant health. Available from Didactico, 2BTrading, and seli.tn for approximately 85.00 TND.
- **Ultrasonic Distance Sensor (HC-SR04):** Used as a water level sensor to monitor water levels in the fish tank and grow beds, preventing overflow or depletion. Available from Didactico, 2BTrading, seli.tn, and Cothings for approximately 5.20 TND each (2 units needed).
- **TDS Sensor (SEN0244 DFROBOT):** To measure the concentration of total dissolved solids in the water, indicating nutrient availability for plants. Available from 2BTrading and seli.tn for approximately 69.00 TND.
- **Photoresistor Module (LDR with LM393):** A light-dependent resistor module. Available from Didactico and Cothings for approximately 5.50 TND.

DESCRIPTION	Didactico	2BTrading	seli.tn	Cothing	QTY	BEST UNIT PRICE	AMOUNT
Capteur de température DS18B20 – sonde de 1 Mètre	https://www.didactico.tn/p	https://2btrading.tn/capte		https://cothings.net/produ	1	6.70TND	6.70TND
KIT CAPTEUR SONDE PH-METRE COMPATIBLE ARDUINO E201-BNC + PH4502C		https://2btrading.tn/acc	https://seli.tn/product/k		1	85.00TND	85.00TND
Capteur de distance à ultrasons HC-SR04	https://www.didactico.tn/p	https://2btrading.tn/accue	https://seli.tn/product/hc-s	https://cothings.net/produ	2	5.20TND	10.40TND
Capteur de qualité de l'eau TDS SEN0244 DFROBOT			https://seli.tn/product/mo		1	69.00TND	69.00TND
AD20P-1230C Pompe à eau submersible			https://seli.tn/product/a	https://cothings.net/prc	1	24.00TND	24.00TND
Pompe à air micro-DC 370 3V CC 500mA	https://www.didactico.tn/p			https://cothings.net/produ	1	9.00TND	9.00TND
Module de photorésistance LDR avec LM393 3Pin sortie Analogique	*https://www.didactico.t*			*https://cothings.net/pro*	1	5.50TND	5.50TND
					TOTAL		209.60TND
Best Option							
Sold Out							
unavailable							

Figure 2: Sensors Prices Estimation

3.3 Wiring Details

The wiring details for the aquaponics system components are as follows:

- **Buck Converter:**
  - **IN+:** Connected to the COM pin of the relay module and VCC of the battery.
  - **IN-:** Connected to GND of the relay module, battery, and water pump.
  - **OUT+:** Powers VCC+ of the relay module, sensors (TDS, pH, temperature), and the Arduino.
  - **OUT-:** Connected to the ground of all sensors and the Arduino.
- **pH Meter:**
  - **Signal:** Connected to A3 on the Arduino.
  - **VCC:** Connected to OUT+ of the Buck Converter.
  - **GND:** Connected to OUT- of the Buck Converter.
- **TDS Sensor:**
  - **AI:** Connected to A0 on the Arduino.
  - **5V:** Connected to OUT+ of the Buck Converter.
  - **GND:** Connected to OUT- of the Buck Converter.
- **Temperature Sensor:**
  - **Temp GND (Black):** Connected to OUT- of the Buck Converter.
  - **Temp VDD (Red):** Connected to OUT+ of the Buck Converter.
  - **Temp DQ (Yellow):** Connected to D5 on the Arduino.
- **1 Channel 5V Relay Module:**
  - **VCC+:** Connected to OUT+ of the Buck Converter.
  - **VCC- (GND):** Connected to IN- of the Buck Converter.
  - **IN:** Connected to D8 on the Arduino.
  - **N.O.:** Connected to the VCC of the water pump.
  - **COM:** Connected to IN+ of the Buck Converter and VCC of the battery.
- **Water Pump:**
  - **VCC:** Connected to N.O. of the relay module.
  - **GND:** Connected to IN- of the Buck Converter.
- **HC-SR04 Ultrasonic Sensor:**
  - **VCC:** Connected to 5V of the Arduino.
  - **TRIG:** Connected to D2 of the Arduino.
  - **ECHO:** Connected to D3 of the Arduino.
  - **GND:** Connected to GND of the Arduino.

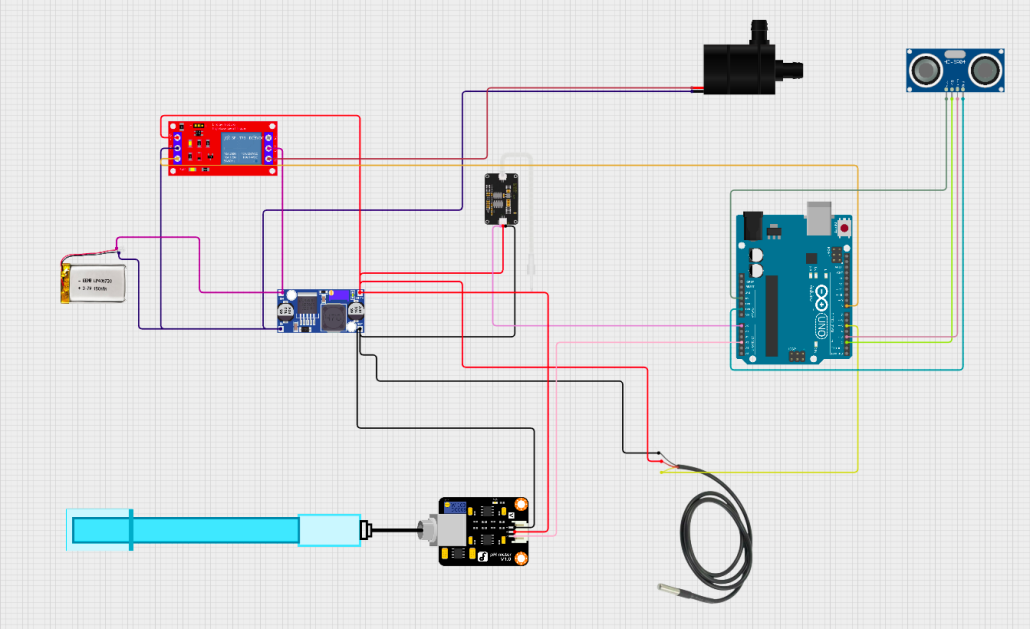

Figure 3: circuit design

4 ChirpStack Server Setup

4.1 ChirpStack Deployment

To explore how ChirpStack works, we deployed it using Docker, simplifying setup and container management.

4.1.1 Steps to Install ChirpStack using Docker

1. Clone the ChirpStack Docker repository:

```
git clone https://github.com/chirpstack/chirpstack-docker.git
cd chirpstack-docker
```

2. Launch ChirpStack:

```
docker-compose up
```

3. Access the web interface:

```
http://localhost:8080
```

For more information, visit the official ChirpStack documentation: https://www.chirpstack.io/docs/getting-started/docker.html

4.2 LWN Simulator for ChirpStack Testing

To test ChirpStack without physical devices, we used the LoRaWAN (LWN) Simulator, which enables virtual gateways and devices.

4.2.1 Requirements

- **Go Language:** Version ≥ 1.21 is required.
- **Windows Users:** Install GnuMake to use the provided Makefile.

4.2.2 Installation Steps

1. Clone the project:

```
git clone https://github.com/UniCT-ARSLab/LWN-Simulator.git
cd LWN-Simulator
```

2. Install dependencies:

```
make
```

3. Build the simulator:

```
make build
```

4. Run the simulator:

```
./lwn-simulator
```

5. Access the web interface:

```
http://localhost:8000
```

4.3 Creating Gateways and Devices

4.3.1 Gateway Configuration

Before creating gateways, configure the Gateway Bridge, the ChirpStack component responsible for LoRa packet forwarding.

4.3.1.1 Gateway Bridge Address

- Use `localhost` for the same machine as ChirpStack.
- For a remote ChirpStack instance, use its IP address.

4.3.1.2 Gateway Bridge Port

- Default: `1700` (Semtech protocol).
- Modify only if the ChirpStack Gateway Bridge configuration differs.

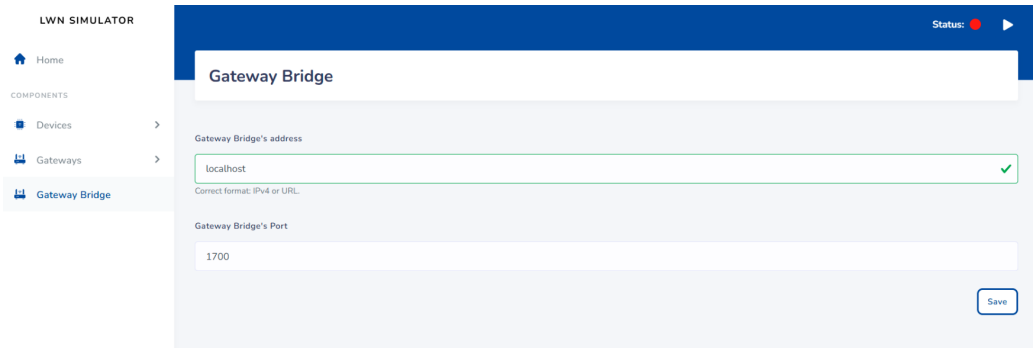

Figure 4: Gateway Bridge Port Configuration

4.3.2 Creating a Virtual Gateway

- **Gateway Name:** Example: `test-gateway`.
- **MAC Address:** Unique 16-digit hexadecimal ID (e.g., `AA:BB:CC:DD:EE:FF:00:01`).
- **KeepAlive Interval:** Default is `30 seconds`.

4.3.3 Creating a Virtual Device

- **Device Name:** Example: `temp-sensor`.
- **DevEUI:** Unique 64-bit identifier.
- **Region:** Frequency band, e.g., `EU868` or `US915`.

Ensure the “Active” box is checked to enable the simulation.

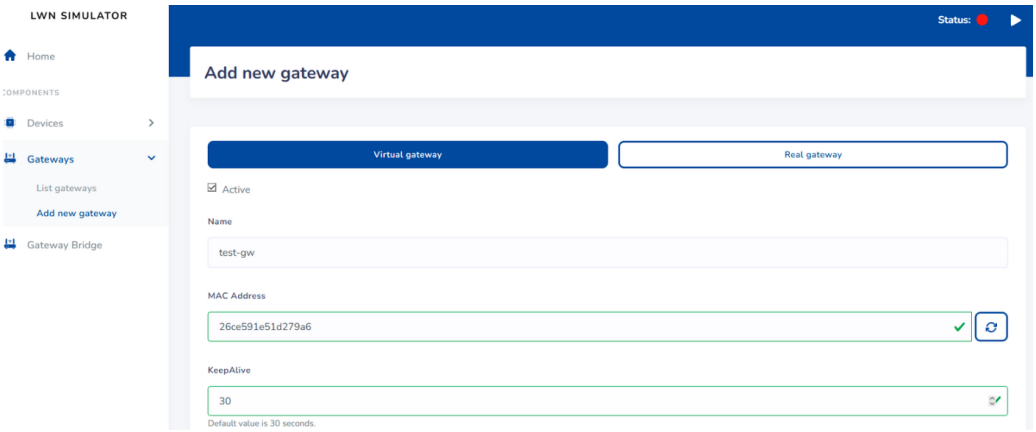

Figure 5: Virtual Gateway Creation

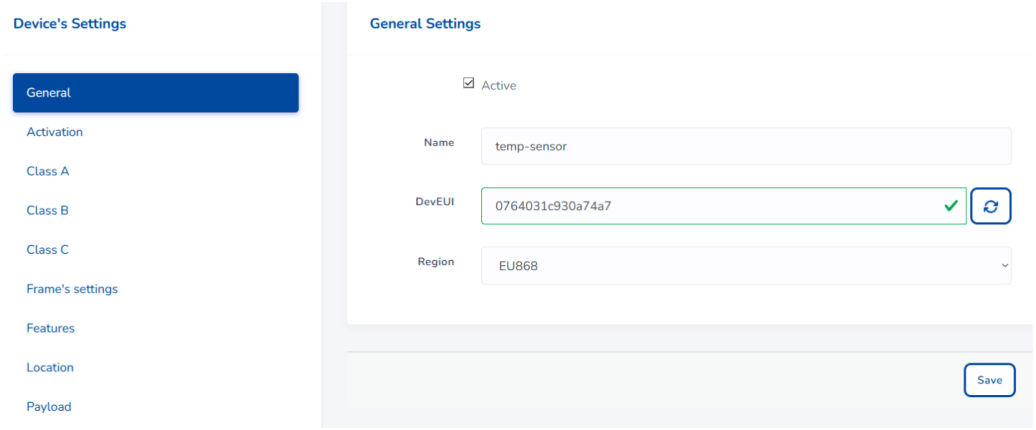

Figure 6: Virtual Device Creation

4.3.4 Activation

4.3.4.1 AppKey

- Automatically generated.
- Used in OTAA (Over-The-Air Activation) to derive session keys.
- Check “OTAA supported” to enable OTAA activation.

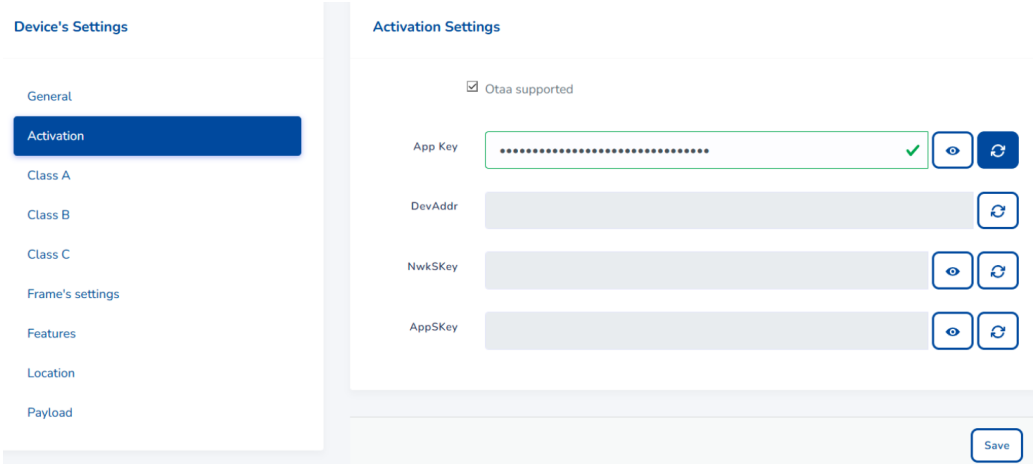

Figure 7: OTAA Activation Settings

4.3.5 Frame Settings

4.3.5.1 Data Rate (DR)

- Defines how fast data is sent (e.g., DR0, DR5).
- Lower DR = longer range, slower speed.

4.3.5.2 FPort

- Frame port for application data.
- FPort 1–223: Application layer.
- FPort 0: Reserved for MAC commands.

4.3.5.3 Retransmission

- Specifies the number of retries if no acknowledgment is received.

4.3.5.4 FCnt (Frame Counter)

- Tracks the number of uplink packets sent.
- Prevents replay attacks by ensuring packet uniqueness.
- Check “Disable frame-counter validation (FCntDown)” for testing purposes. This prevents the server from rejecting messages due to mismatched counters.

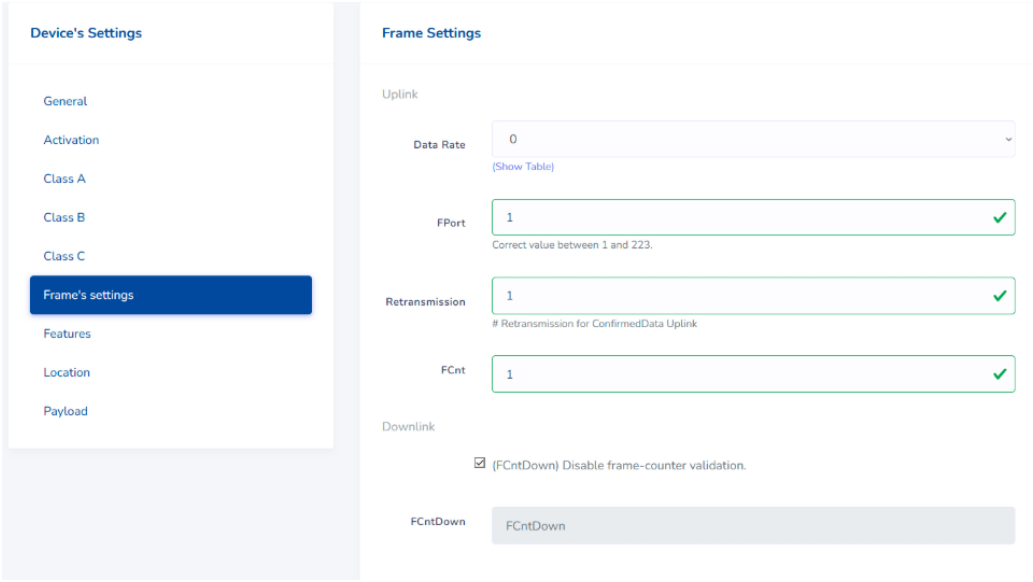

Figure 8: Frame Settings Configuration

4.3.6 Payload

4.3.6.1 Uplink Interval

- Specifies the time between simulated messages (e.g., 30 seconds).

4.3.6.2 Payload

- Represents the actual data being sent.

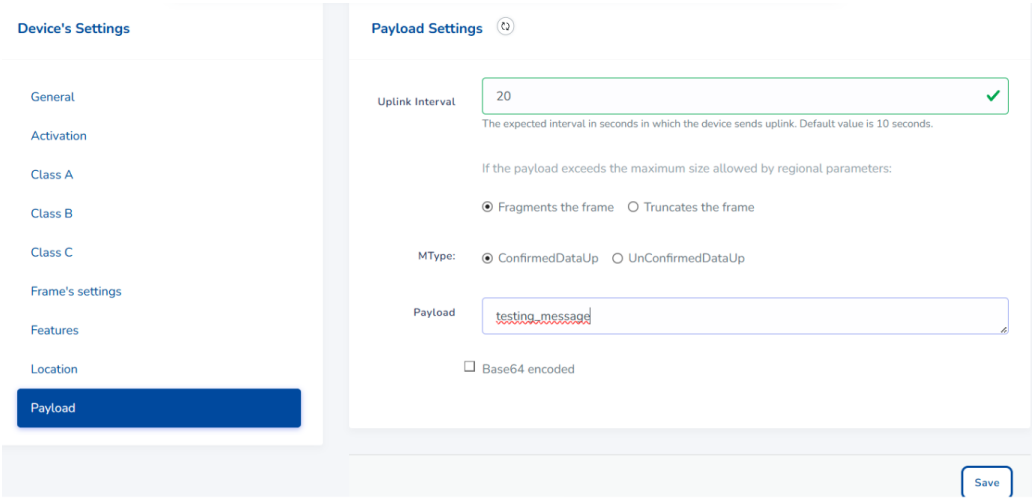

Figure 9: Payload Configuration

4.4 Configuration of ChirpStack

- **Step 1: Create a Tenant**
  - A tenant is like a workspace or client account in ChirpStack. It allows you to separate users, devices, and data under different organizations.

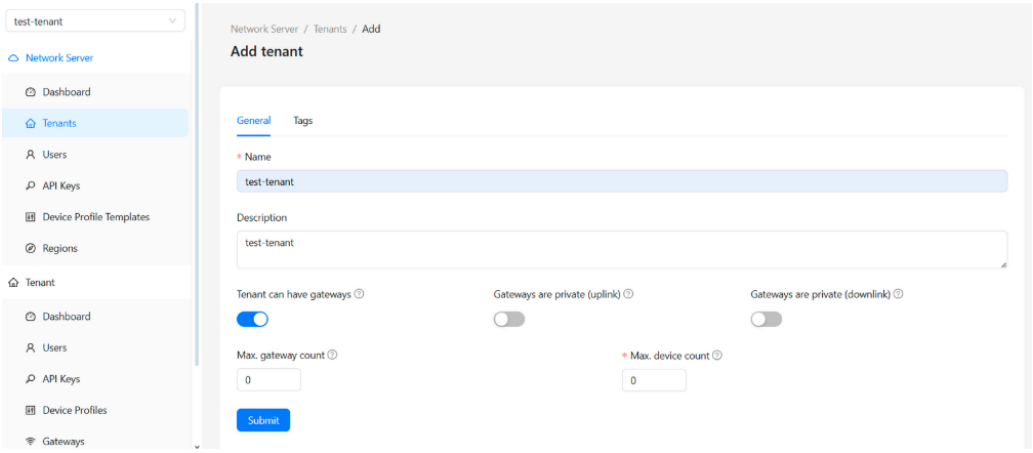

Figure 10: Create a Tenant

- **Step 2: Create a Device Profile**
  - A Device Profile tells ChirpStack how to communicate with a specific type of device (LoRaWAN version, class, frequency region, timing, payload codecs, etc.).

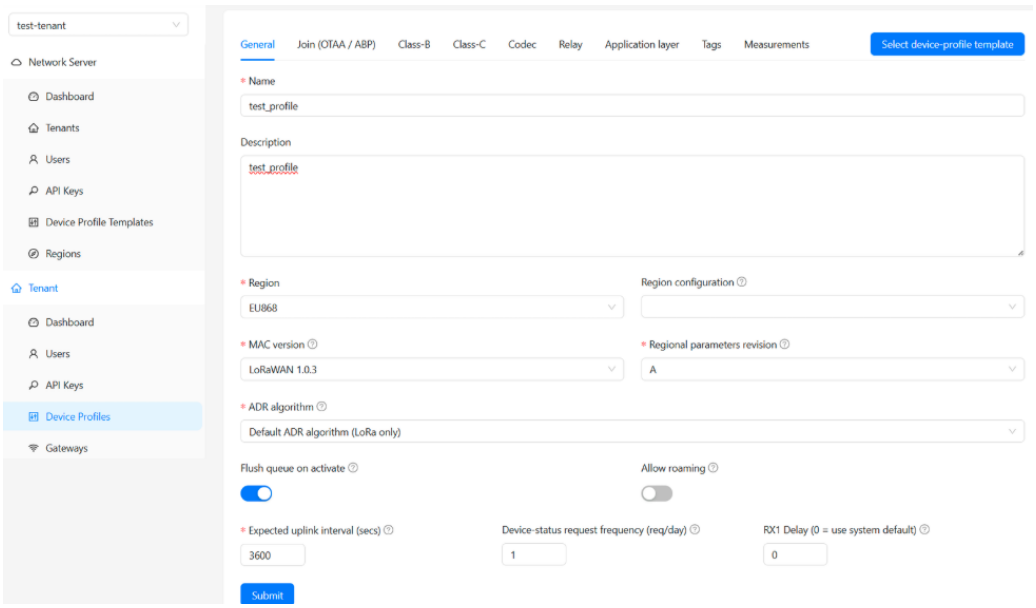

Figure 11: Create a Device Profile

- **Step 3: Add a Gateway**
  - This is where you register your physical or virtual gateway, so ChirpStack knows how to route messages it receives from it.
    (Gateway ID → (copy MAC from LWN gateway))

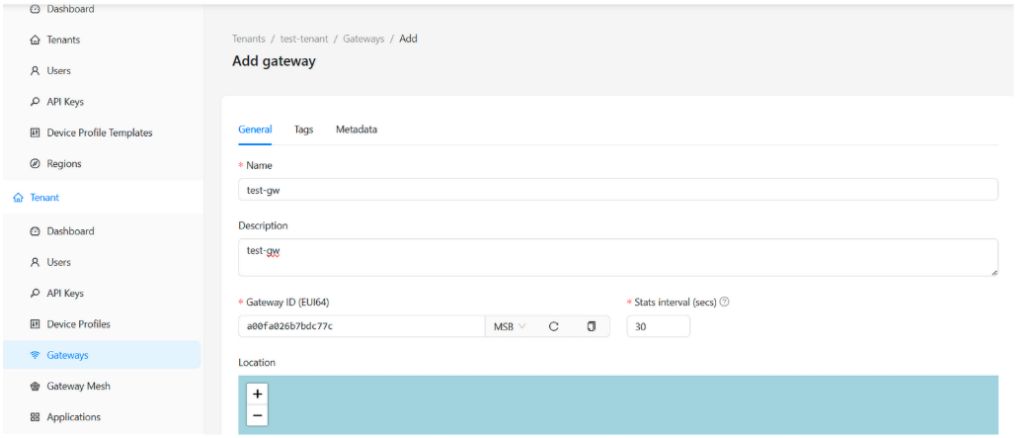

Figure 12: Add a Gateway

item **Step 4: Add an Application**

- An Application is a logical container that groups devices that send similar types of data. It also allows you to define integrations (e.g., HTTP, MQTT) for those devices.

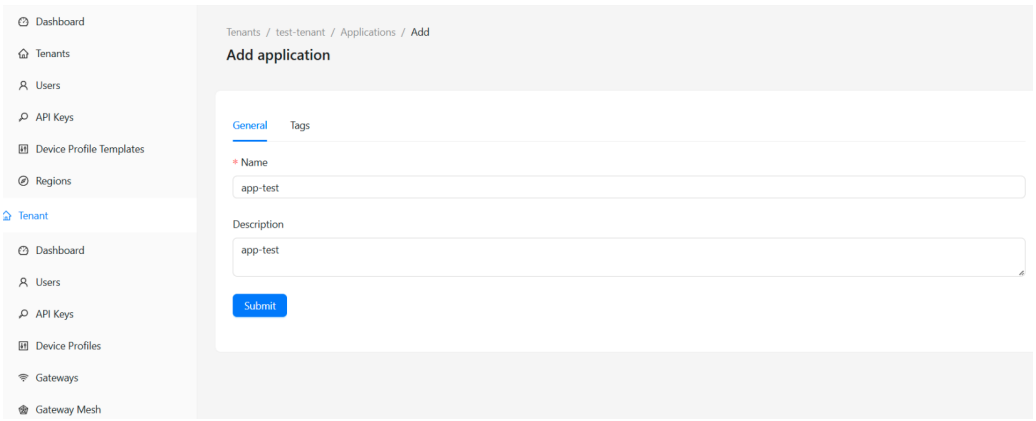

Figure 13: Add an Application

item **Step 5: Add a Device**

– Device EUI must match the DevEUI from the LWN simulator.
Paste the AppKey (from LWN simulator).

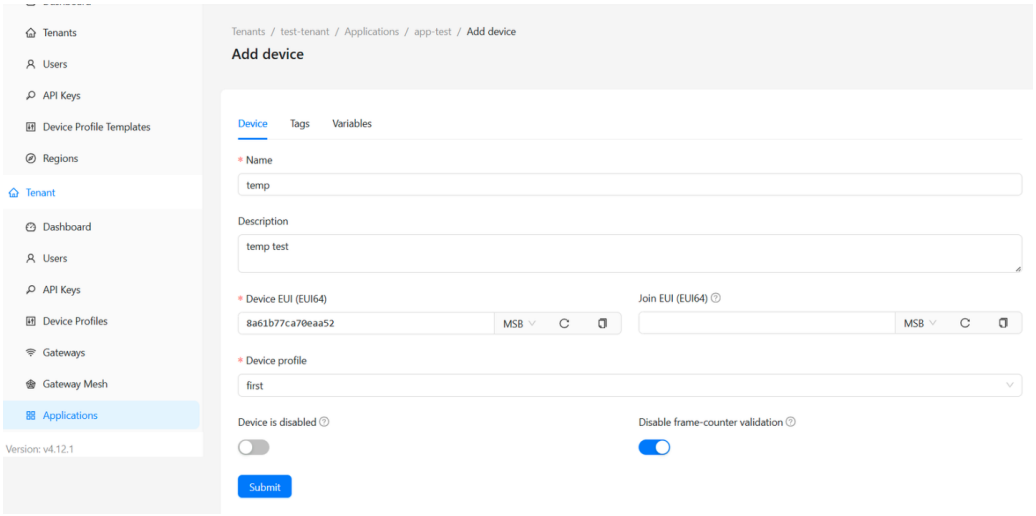

Figure 14: Add a Device

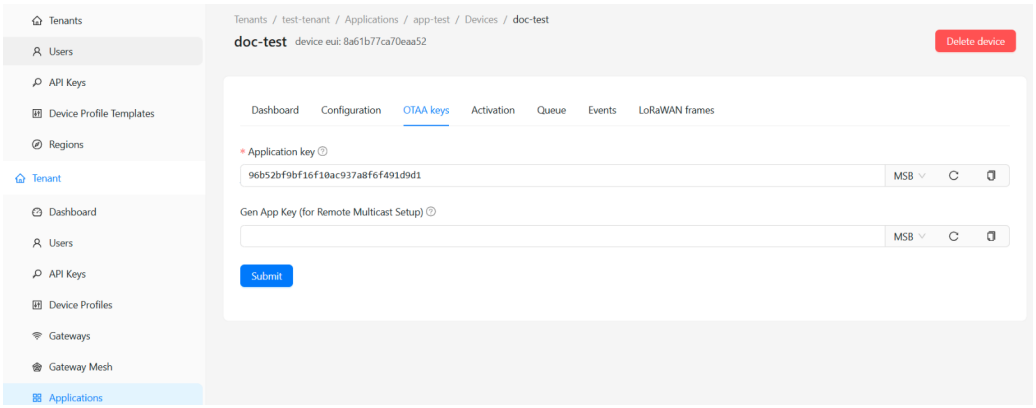

Figure 15: Paste the AppKey (from LWN simulator)

4.5 Integration via HTTP Webhook

To integrate ChirpStack with our web application, we added an HTTP Integration.

4.5.1 Integration Steps

1. In the Application settings, navigate to the **Integrations** tab.
2. Add an HTTP endpoint.

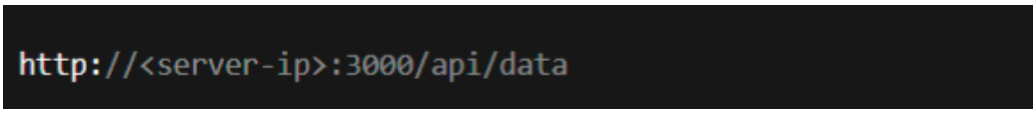

Figure 16: HTTP endpoint

3. Set the URL to point to our backend server.

4.5.2 Backend – Receiving and Processing Payloads

Once integrated, ChirpStack sends a POST request with JSON payloads every time a device transmits data.

On the backend, we implemented a Node.js + Express server to handle the payloads. The server is responsible for receiving, decoding, and storing the incoming payloads in a MongoDB database. It exposes two main endpoints:

- **POST Endpoint:**

```
app.post("/api/data", (req, res) => {
    // Process incoming data
    const payload = req.body;
    // Store payload in the database
    database.save(payload);
    res.status(200).send("Data␣received");
});
```

This endpoint handles incoming POST requests from ChirpStack, receiving payload data.

- **GET Endpoint:**

```
app.get("/api/data", async (req, res) => {
    const devices = await database.getUniqueDevices();
    const result = {};
    for (const device of devices) {
        result[device] = await database.getLast10Entries(device);
    }
    res.json(result);
});
```

This endpoint retrieves the last 10 JSON records for each device. It first identifies all unique device names and then queries the 10 most recent entries for each, grouped by device.

4.5.3 Testing Payloads

The backend also supports payloads in the form `random(x,y)`, which generate a random value between `x` and `y` for testing purposes before storing them in the database.

5 Testing Phase

5.1 Testing Phase: Verifying the LoRaWAN-Based Aquaponics System

The testing phase focused on validating the communication and functionality of the LoRaWAN-based aquaponics system, emphasizing integration with ChirpStack using Docker. The key components involved were:

- **Raspberry Pi with SX1276 LoRa HAT:** Configured as a single-channel LoRaWAN gateway for receiving data packets from sensor nodes.
- **Arduino Mega 2560 with SX1276 LoRa Shield:** Functioning as a sensor node for collecting data from the TDS sensor and transmitting it via LoRaWAN.
- **TDS Sensor (SEN0244 DFROBOT):** Used to measure the concentration of total dissolved solids in water, critical for assessing water quality.

5.1.1 Testing Procedure

1. **Enabling SPI on Raspberry Pi:**
   - SPI was activated to allow the Raspberry Pi to communicate with the LoRa HAT.
   - Steps:
     - Access the menu using `sudo raspi-config`.
     - Navigate to **Interfacing Options** > **SPI** > **Enable**.
     - Reboot the Raspberry Pi using

```
sudo reboot
```

2. **Installing and Configuring the Single-Channel Packet Forwarder:**
   - Essential tools were installed:

```
sudo apt update
sudo apt install git build-essential
```

   - Installed `wiringPi` using the GitHub repository:

```
git clone https://github.com/WiringPi/WiringPi.git
cd WiringPi
./build
```

   - Downloaded the Single-Channel Packet Forwarder:

```
git clone https://github.com/m2mlorawan/single_chan_pkt_fwd
cd single_chan_pkt_fwd
```

- Configured the forwarder in `global_conf.json`:
  - Frequency: `868100000 Hz`.
  - Gateway ID: Unique identifier.[in this configuration the gateway EUI is > E45F01FFFFEA86AF]
  - Server: `or IP [in this configuration we used 192.168.201.13]`
  - UDP Port: `1701:1700`.
- Compiled the forwarder using:

```
make
```

- Launched the forwarder with:

```
sudo ./single_chan_pkt_fwd
```

3. **Configuring ChirpStack Gateway Bridge:**
   - Installed ChirpStack Gateway Bridge:

```
sudo apt install chirpstack-gateway-bridge
```

   - Edited the configuration file (`/etc/chirpstack-gateway-bridge/chirpstack-gateway-bridge.toml`):

```
[integration.mqtt]
server = "tcp://192.168.201.13:1883"

[backend]
type = "semtech_udp"
bind = "0.0.0.0:1701"
       \end{verbatim}
       \item Restarted the Gateway Bridge service:
       \begin{verbatim}
sudo systemctl restart chirpstack-gateway-bridge
```

```
sudo systemctl restart chirpstack-gateway-bridge
```

4. **Setting Up ChirpStack with Docker:**
   - Modified ChirpStack Docker ports:
     - **UDP Port:** `1701:1700`.
     - **HTTP Port:** `9090:8080`.
5. **Sensor Node Setup:**
   - Connected the TDS sensor to the Arduino Mega.
   - Processed sensor data and formatted it into LoRaWAN packets for transmission.
6. **Data Transmission and Reception:**
   - Test messages containing TDS data were transmitted from the Arduino Mega.

- The Raspberry Pi gateway successfully received and displayed the data in the ChirpStack web interface.

7. **Error Handling:**
    - Simulated scenarios, including packet loss and frequency mismatches, to evaluate system resilience.

5.1.2 Results

- LoRaWAN communication between the sensor node and gateway was reliable over short to medium distances.
- ChirpStack processed and displayed the received data correctly.
- The TDS sensor provided accurate readings that were successfully transmitted.
- The system is ready for scaling with additional sensors and deployment in more extensive aquaponics setups.

5.2 Arduino Code for LoRa Sender with TDS Sensor

The following Arduino code configures the sensor node to transmit TDS (Total Dissolved Solids) data via LoRa:

Listing 1: Arduino Code for LoRa Sender with TDS Sensor

```
#include <SPI.h>
#include <LoRa.h>

#define TDSPIN A1
#define SS_PIN 10
#define RST_PIN 9
#define DIO_PIN 2
#define LORA_FREQUENCY 868100000

void setup() {
  Serial.begin(9600);
  while (!Serial);

  Serial.println("LoRa␣Sender␣with␣TDS␣Sensor");

  LoRa.setPins(SS_PIN, RST_PIN, DIO_PIN);

  if (!LoRa.begin(LORA_FREQUENCY)) {
    Serial.println("Starting␣LoRa␣failed!");
    while (1);
  }
  Serial.println("LoRa␣initialized␣successfully!");
  LoRa.setSpreadingFactor(7);
  LoRa.setSignalBandwidth(125E3);
  LoRa.setCodingRate4(5);
}

void loop() {
  int sensorValue = analogRead(TDSPIN);
  float voltage = sensorValue * (5.0 / 1024.0);

  float tdsValue = (133.42 * voltage * voltage * voltage -
                  255.86 * voltage * voltage +
                  857.39 * voltage) * 0.5;

  Serial.print("TDS␣Sensor␣Value:␣");
  Serial.print(sensorValue);
  Serial.print(",␣Voltage:␣");
  Serial.print(voltage);
  Serial.print("V,␣Estimated␣TDS:␣");
  Serial.print(tdsValue);
  Serial.println("␣ppm");

  String message = "TDS:" + String(tdsValue) + ";";
```

```
  Serial.print("Sending packet: ");
  Serial.println(message);

  LoRa.beginPacket();
  LoRa.print(message);
  LoRa.endPacket();

  Serial.println("Packet sent!");

  delay(5000);
}
```

5.2.1 Screenshots of Data Transmission and Reception

Figures 17 and 18 illustrate the successful transmission and reception of TDS data using the LoRaWAN system.

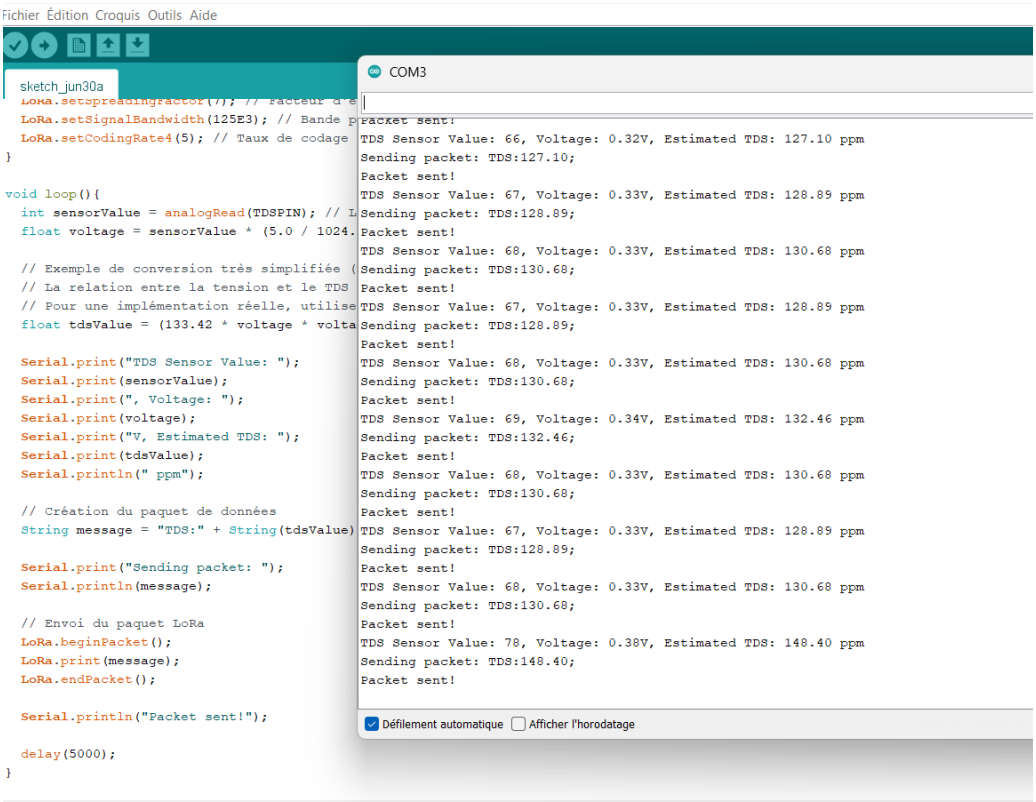

Figure 17: Data transmission screenshot from the Arduino serial monitor.

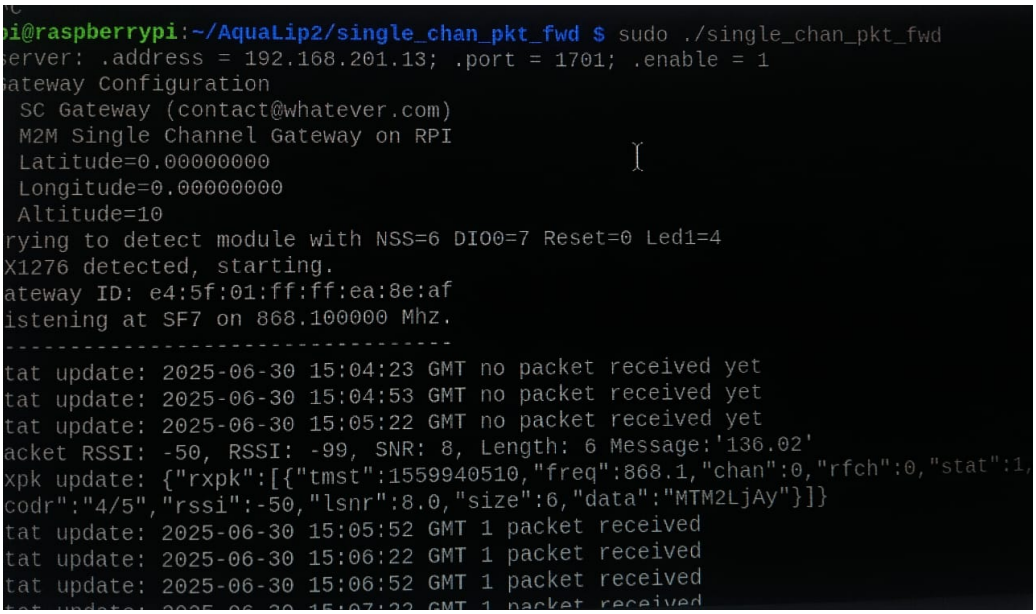

Figure 18: Data reception screenshot from the Raspberry Pi terminal.

6 Website and Dashboard Development

6.1 Project Overview and Demo Website

The project website serves as the central public platform for the Mini Aquaponic System with LoRaWAN Implementation. It is designed to:

- Present the project's goals and sustainability context.
- Explain the principles of aquaponics and smart agriculture.
- Demonstrate real-time system capabilities through interactive elements.

Tools and Technologies Used:

- **Frontend:** HTML5, CSS3, JavaScript, React.ts
- **Backend:** Node.js (for REST APIs), Express
- **Data Visualization:** Chart.js, Recharts
- **IoT Network:** ChirpStack (LoRaWAN network server)
- **Hosting:** GitHub Pages

The website contains multiple sections:

- **Aquaponics Explained:** Introduction to aquaponic farming, including diagrams and environmental benefits.
- **System Architecture:** Technical schematics showing sensor placement, LoRaWAN data flow, and dashboard interaction.
- **Live Dashboard Access:** Embedded sensor dashboard (see below).
- **Team and Contributions:** Credits and roles of each team member.

Figure: Screenshot of the Demo Website Homepage

6.2 Sensor Data Dashboard

The sensor data dashboard is a core feature, enabling users to visualize, monitor, and analyze real-time and historical data collected from the aquaponic system. It enhances transparency and control over system conditions.

Dashboard Features:

- **Real-time Monitoring:** Display of live sensor readings including pH, dissolved oxygen (DO), water level, and temperature.
- **Historical Trends:** Interactive line and bar graphs for analyzing data trends over time.

- **Threshold Alerts:** Automated alerts triggered when values exceed or fall below defined thresholds.
- **Mobile Responsive UI:** Optimized for desktops and mobile devices for remote access.

Technical Integration:

- Sensor data is transmitted via LoRaWAN to the ChirpStack network server.
- ChirpStack forwards decoded payloads to an MQTT broker or HTTP integration.
- A backend service consumes this data, stores it in a time-series database (e.g., InfluxDB), and exposes it via REST APIs.
- The frontend or Streamlit dashboard fetches and visualizes this data using dynamic charting libraries.

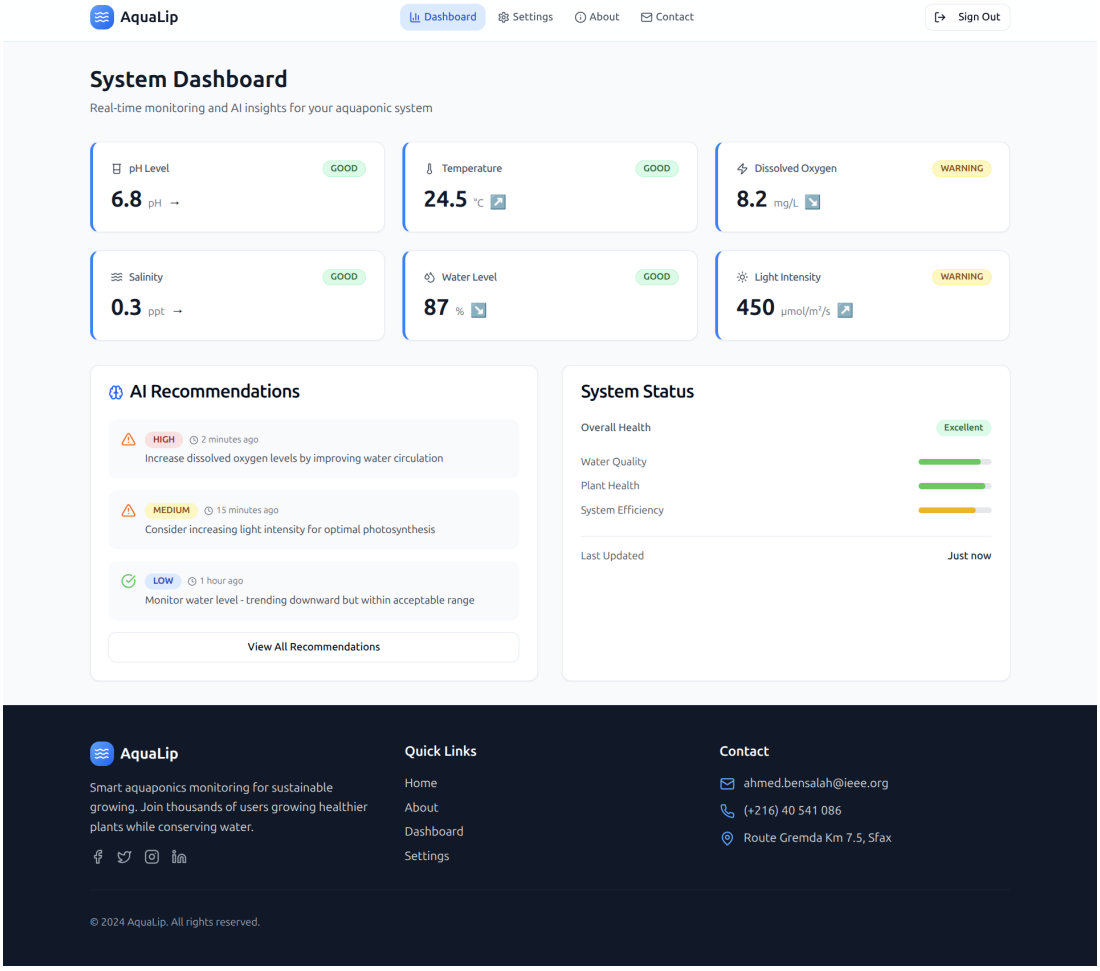

Figure: Live Sensor Dashboard showing pH, DO, Temperature and Water Level

The real-time dashboard empowers users to maintain ideal environmental conditions, promoting healthier ecosystems for both fish and plants and maximizing productivity in a sustainable manner.

7 Conclusion

This internship project successfully developed and implemented a mini aquaponic system enhanced with LoRaWAN technology for smart monitoring and control. The integration of various sensors, Arduino microcontrollers, Raspberry Pi gateways, and the ChirpStack network server has created a robust and efficient solution for sustainable food production. The accompanying website and dashboard provide a user-friendly interface for project overview and real-time data visualization, demonstrating the practical application of IoT in agriculture.

Future work could involve expanding the system to a larger scale, integrating more advanced control mechanisms (e.g., automated nutrient dosing), and exploring machine learning algorithms for predictive analysis of system health. The success of this project at the WEDTECT Startup, stemming from the I2I hackathon, underscores the potential of innovative technological solutions in addressing contemporary challenges in food sustainability.

References

[1] MDPI reviewers. “Aquaponics: A Sustainable Path to Food Sovereignty and Enhanced Water Use Efficiency”. In: *Water* ().

[2] The Things Network. *What are LoRa and LoRaWAN?* https://www.thethingsnetwork.org/docs/lorawan/what-is-lorawan/. Accessed Jun.2025.

[3] Wikipedia contributors. “LoRa”. In: *Wikipedia, The Free Encyclopedia* (2025). Accessed Jun.2025.